Project Wealthy Minds

Higher National Diploma in Software Engineering

School of Computing and Engineering

National Institute of Business Management Colombo-7

PROJECT MEMBERS:

U.G.D.S. Karunathilake (COHNDSE252F-026)

T.N.V. Perera (COHNDSE243F-065)

Ravindu K.A.D.C(COHNDSE252F-001)

Student Index Number: COHNDSE252F-026

NIBM Registered Name: U.G.D.S. Karunathilake

Programme: Higher National Diploma in Software Engineering

Batch: HNDSE 25.2F

Selected Data Structure: BST Implementation

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

1. Introduction

As we know managing can be challenging for students and young adults, Especially when it comes managing money people don't do accurately as they think when they are doing it manually So that is why Wealthy Minds was proposed as a personal financial managing system that provides all the basic income and expense tracking as well as this application provides users to keep track of their savings, investments, loans, subscriptions, make goals and also generate financial reports at the end and also provide a Financial Health Score. Through these, Wealthy Minds aims to help people make smarter financial decisions and overall be smarter when making decisions about money.

2. Problem Statement

Although many finance applications track transactions, subscriptions and account balances they often show users what happened without clearly explaining what they should do next. Wealthy Mind addresses this gap by organizing financial information in one place and turning it into useful insights, alerts and recommendations. The Binary Search Tree is therefore used for a practical purpose, storing, searching and retrieving transaction's efficiently and having the data in order.

3. Literature Review

While popular platforms like YNAB offer robust tools for setting savings targets and planning debt repayments (YNAB, 2025, 2026), Rocket Money excels at subscription tracking with premium add-ons (Rocket Money, 2026), and Empower provides a high-level view of overall net worth and investments (Empower, 2026), these apps mostly focus on logging past activity. Wealthy minds do more than simply record financial data. It helps users understand what their spending habits mean by providing a clear financial health score, warning them about risks and offering insightful information by an efficient data structure.

Table 1. Existing tools and identified gap

Existing tool	Established capability	Gap addressed by Wealthy Minds
YNAB	Targets and loan scenarios	Unified behavioural score and explainable prediction
Rocket Money	Budget and subscription monitoring	Transparent prioritisation logic and education
Empower	Cash flow and portfolio dashboard	Data-structure-driven relationship analysis

4. Proposed Solution

In this module, each transaction is stored as a node in a BST using its data and transaction ID as a unique key. Smaller keys are placed on the left and larger keys on the right, making it easier to insert, search for and remove financial records. An in-order traversal can then retrieve the transactions in-order without requiring a separate sorting processes all records in O(n) time. One limitation is that the tree can become unbalanced when transactions are inserted in an already sorted sequence, causing operations to take O(n) time in worst case. Despite this limitation, a standard BST is suitable because of its ordering rules, recursive operations and traversal methods are clear to implement, test and explain.

5. System Functionalities

- Track Every Dollar: Easily log all incoming and outgoing money—including income, daily expenses, savings, investments, active loans, recurring subscriptions, and long-term financial goals.
- Flexible Financial Reporting: Automatically generate monthly, quarterly, and annual statements alongside broken-down category summaries without needing extra sorting.
- Transparent Health Scoring: Compute a real-time Financial Health Score that doesn't just give a raw number, but actually explains the specific habits driving it.
- Proactive Forecasting & Alerts**:** Forecast upcoming expenses, flag potential overspending early, and offer practical steps to get back on track before money runs tight.

Table 2. Input-process-output mapping

Input	Processing	Output
Transaction, amount, category, date	Validation, storage and selected-structure operation	Searchable financial record
Accounts, loans and goals	Behaviour analysis and health scoring	Score, trend and explanation
Historical spending	Forecasting and threshold checks	Prediction, alert and recommendation

6. Selected Data Structure and Justification

A Binary Search Tree was selected because it keeps financial transactions organized while supporting efficient insertion, searching and deletion. When the tree remains reasonably balanced, these operations have an average time complexity of O (log n), while an in-order traversal processes all records in O(n) time in the worst case. Except for this a standard BST is suitable beacuase of its rules and methods are clear to implement, test and explain.

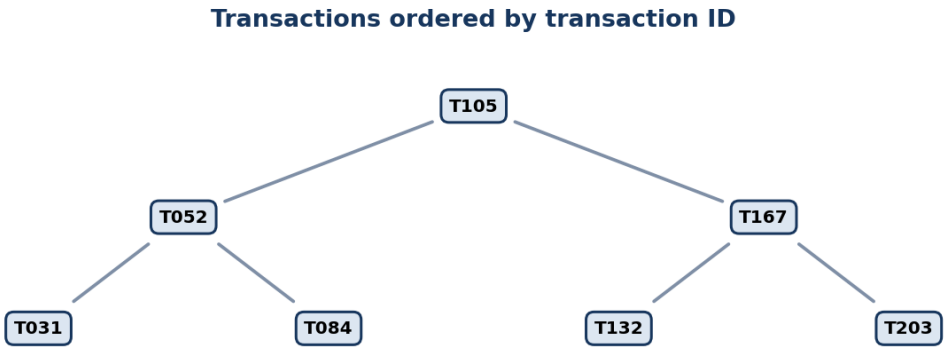

Figure 1. BST arrangement of financial transactions

Table 3. Core operation complexity

Operation	Expected / typical	Worst case
Insert transaction	O(log n)	O(n)
Search by key	O(log n)	O(n)
Delete transaction	O(log n)	O(n)
In-order statement	O(n)	O(n)

7. Novel Features

- Automatic Financial Statements and Report Generations
- Spending Pattern Detections
- Smart Transaction Search
- Category-Based Range Analysis

8. Technologies Used

Here we selected Java was selected for the prototype because its object-oriented approach makes it easier to organize the system into classes such as Transaction, BSTNode, TransactionBST, FinancialAnalyzer and Report Service. JavaFX or Swing can be used for the interface, while JSON or CSV files handle data transfers. JUnit helps test the BST operations. Building a custom BST also makes the logic easier to understand.

9. Selling Points of the Application

- Provides useful financial advice and early spending warnings.
- Clearly explains how financial scores and insights are calculated.
- Offers a free and accessible solution for students and young adults.
- Uses the BST to support practical transaction management.

10. Individual Reflection

Learning Journey

Working on Wealthy Minds helped me understand how a BST can be used outside the classroom. By developing the transaction module, I learned how nodes, ordering rules and traversals can organize and retrieve financial records efficiently. The project also improved my understanding of planning, testing and using GitHub during software development.

Key Takeaways

- Improved my understand of BST insertion, searching, deletion and traversal.
- Learned how Big-O complexity relates to real system performance.
- Developed better GitHub, teamwork and project documentation skills.
- Learned to design features based on the user's needs.

Development Process

I started by modeling the Transaction and BSTNode objects, then built out the core tree operations. To generate sorted financial statements, I used an in-order traversal. Taking a step-by-step approach helped me lock down a stable base before trying to add extra reporting tools, which kept the codebase readable and easy to test.

Challenges & How I Solved Them

- **Handling Duplicate Keys:** Transactions often share the same timestamp, so I created a unique key by combining the date with the transaction ID.
- **Tricky Deletions:** Node deletion gets messy fast. I split my test cases into leaf, single-child, and two-child scenarios to make sure I wasn't accidentally breaking the tree pointers.
- **Tree Imbalance:** I realized early on that a standard BST can degrade into $O(n)$ if the data comes in sorted. I noted this straight away in the docs as a key area for future improvement (like upgrading to an AVL or Red-Black tree).

Repository Links

- **GitHub Profile:** [Daham728 (Daham Karunathilake)
- **GitHub Repository:** [Daham728/Wealthy Minds]

11. Conclusion

The custom BST helps Wealthy Minds organize, search and retrieve financial transactions efficiently. It also shows how a data structure can support a practical feature instead of being used only for academic purposes. Although an ordinary BST can become unbalanced, it provides a strong foundation that can later be improved using a self-balancing tree. Overall, the system goes beyond basic expense tracking by providing useful financial insights, scores and early warnings.

Implementation Validation Plan

- Unit-test normal, empty, duplicate and boundary inputs.
- Verify every operation against a small hand-calculated dataset.
- Measure execution time as data volume increases and compare with stated complexity.
- Conduct task-based usability testing for data entry, insight discovery and error recovery.

References

Empower. (2026). Budget & cash flow planner. https://www.empower.com/tools/budgeting-cash-flow

JGraphT. (n.d.). DijkstraShortestPath. https://jgrapht.org/javadoc/org.jgrapht.core/org/jgrapht/alg/shortestpath/DijkstraShortestPath.html

National Institute of Standards and Technology. (2020). The on-line dictionary of algorithms and data structures (NIST IR 8318). https://doi.org/10.6028/NIST.IR.8318

Oracle. (n.d.). PriorityQueue (Java Platform SE 8). https://docs.oracle.com/javase/8/docs/api/java/util/PriorityQueue.html

Oracle. (n.d.). TreeMap (Java Platform SE 8). https://docs.oracle.com/javase/8/docs/api/java/util/TreeMap.html

Rocket Money. (2026). What is Rocket Money and why should you sign up? https://www.rocketmoney.com/learn/personal-finance/what-is-rocket-money

YNAB. (2025). Slay your loans with YNAB's loan payoff simulator. https://www.ynab.com/blog/ynab-loan-planner

YNAB. (2026). Features. https://www.ynab.com/features